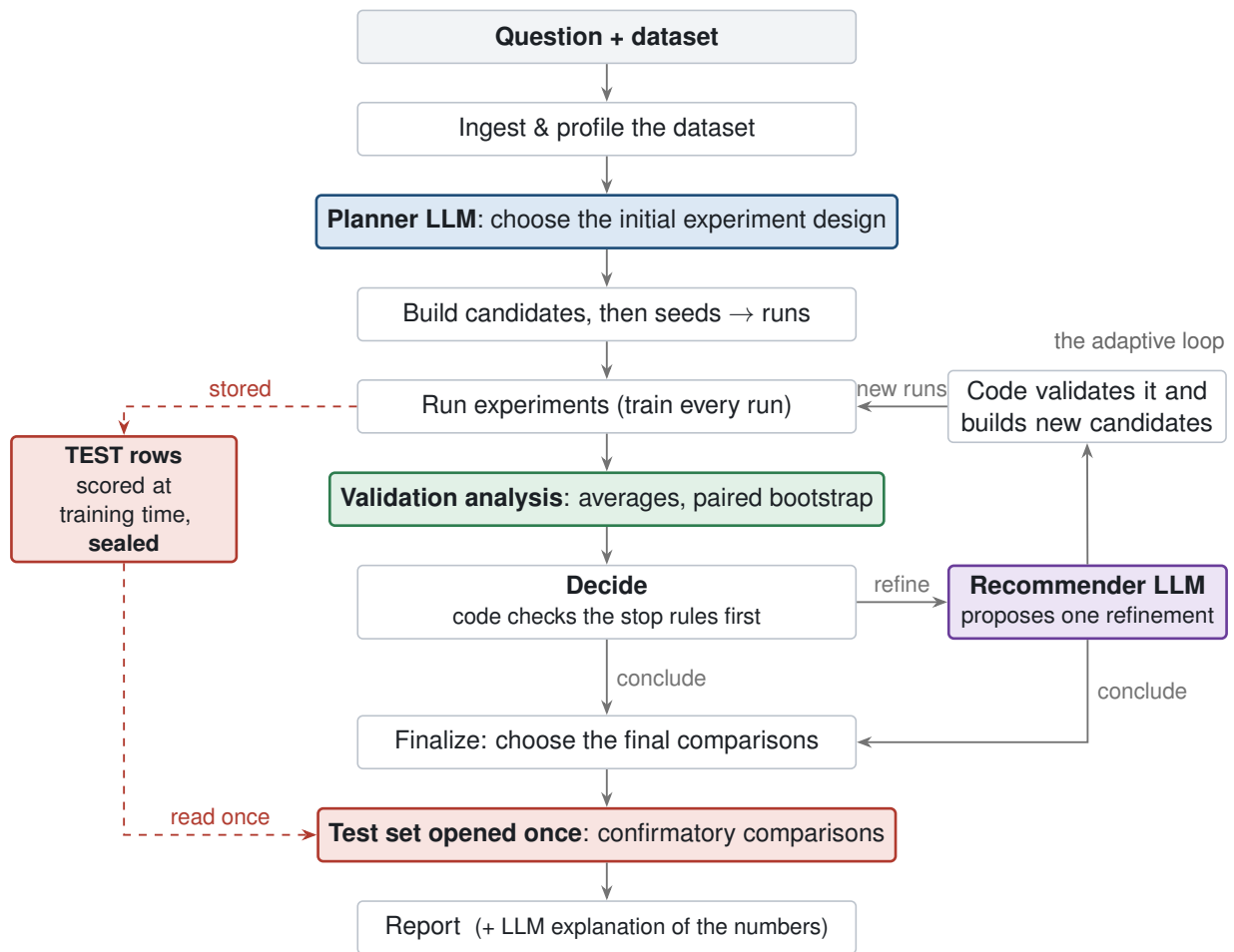


How the Adaptive ML Experiment Agent answers a question

A research question and a CSV go in; a statistically honest answer comes out. This page is the whole idea. The rest of the document opens each box.



Every rule and every number: candidates, seeds, training, statistics, stop rules, validation of LLM proposals.

Planner LLM

One call. Reads the question and dataset *counts*; returns the initial design. Trains nothing, computes nothing, never sees a data row.

Recommender LLM

At most one call per round. Reads a compact summary of validation results; *recommends* one refinement or stopping. Code decides whether it is allowed.

validation

The only data the adaptive loop ever looks at. Paired bootstrap happens here, every round.

sealed test

Scored when each model is trained, then sealed. Read exactly once, after the loop has stopped and the comparisons are fixed.

Two kinds of question, one skeleton. Selection Mode asks *which model family is best?* — the initial design is several model families, and refinement tunes one knob of a family still in the race. **Effect Mode** asks *does factor X matter?* — the initial design is several levels of one factor, and refinement only adds more levels of that same factor. Training, seeds, the bootstrap and the sealed test set work identically in both.

2 Common setup — the same for both modes

A Ingest & profile

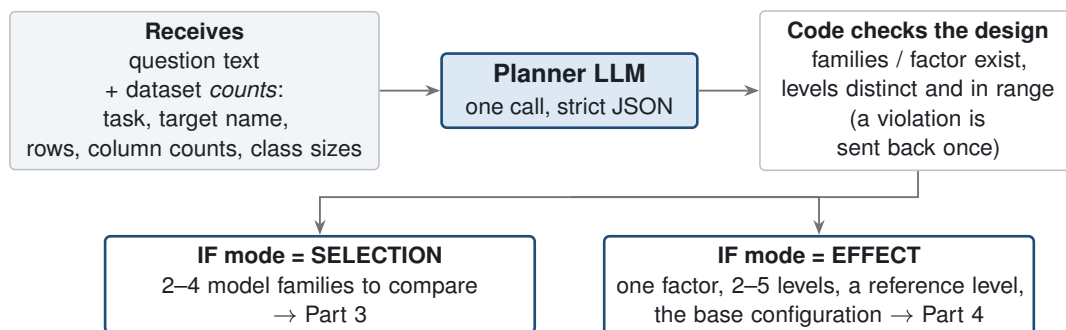
- **In:** the CSV, the target column the user names (never guessed), the question.
- Rows with any missing value are dropped (no imputation).
- Task type: text target, or ≤ 20 whole-number values → classification; otherwise regression.
- Categorical columns are one-hot encoded.
- One fixed split for every run, ever: **70 % train / 15 % validation / 15 % test**.
- **Out:** a DatasetProfile (counts, classes, split seed).

Real run — diabetes_risk.csv

15 000 rows → 10 500 after dropping missing values
target diabetes_risk: Low / Moderate / High → classification
18 feature columns → 49 after one-hot
split: 7 350 train / 1 575 validation / 1 575 test
always predicting “Low” scores 60.0 % (the sanity floor)

B Planner LLM — the initial design

Planner LLM



The Planner does NOT

- train models or run experiments,
- calculate any statistic,
- see individual data rows or column names.

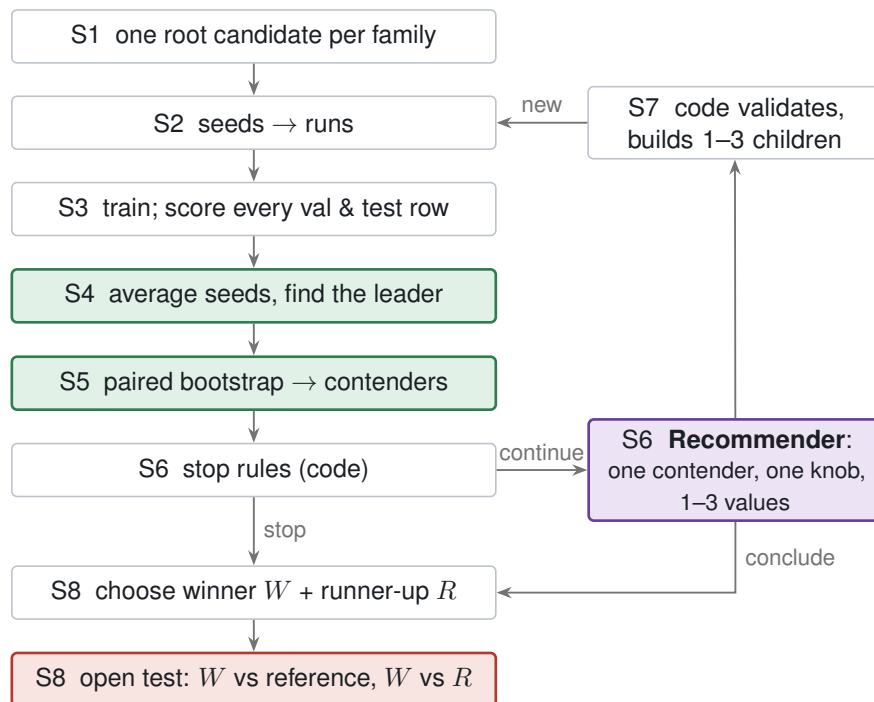
The four model families it can choose from

logistic / linear regression (linear_baseline), decision tree, random forest, MLP (one hidden layer). Each has fixed defaults and allowed ranges in a code registry.

Candidate $\neq$ **run**. A **candidate** is one configuration — model family + hyperparameters + preprocessing — *without* a random seed. A **run** is one actual training of a candidate with one seed. Random forests and MLPs depend on the seed, so each of their candidates is trained **3 times** (seeds 0, 1, 2). Logistic regression and the decision tree do not, so they are trained **once**. Every statistic is computed per *candidate*, after averaging its runs.

3 Selection Mode — “which model family is best?”

Running example (real run): “Which model performs best for classification on this dataset?”; the Planner chooses all four families.



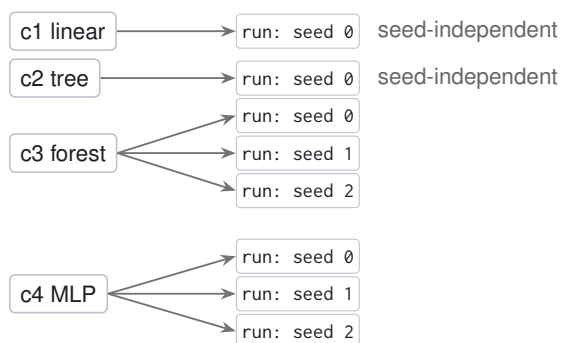
S1 Create root candidates

Each chosen family gets exactly **one** root candidate at its registry defaults. The simplest chosen family (order: linear < tree < forest < MLP) becomes the **reference** — the baseline every other candidate is measured against.

```
c1 = linear_baseline  
c2 = decision_tree (max_depth 8)  
c3 = random_forest (200 trees)  
c4 = mlp (hidden 64, 20 epochs, features normalized)
```

reference

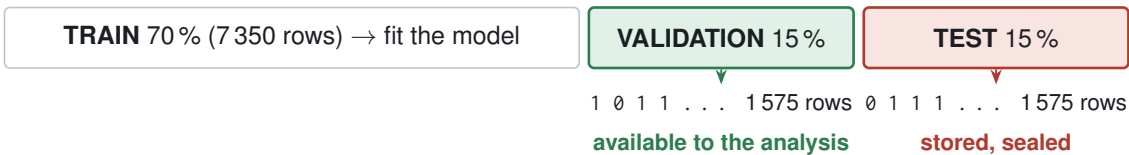
S2 Expand candidates into runs



4 candidates → 8 actual training runs.

A seed changes only randomness *inside* training (weight initialisation, dropout masks, batch order, a forest’s bootstrap samples). It never changes which rows are train, validation or test.

S3 Train — and score every row



Each run scores **every** validation row and **every** test row — not just one accuracy number:

$$\text{score}(i) = \begin{cases} 1 \text{ correct, } 0 \text{ wrong} & \text{classification} \\ (\hat{y}_i - y_i)^2 & \text{regression} \end{cases}$$

It also records the same metric on the *training* rows (to spot overfitting), the training loss and the time. A run that crashes or produces NaN/Inf is marked failed; every other run counts as evidence. Each run is saved as it finishes.

S4 Validation analysis — average the seeds, find the leader

validation

First, seeds are averaged row by row (illustration)

	row 1	row 2	row 3	row 4	...
forest seed 0	1	0	1	1	
forest seed 1	1	1	1	0	
forest seed 2	0	1	1	1	
c3 forest	.67	.67	1	.67	

⇒ one validation score *per row* per candidate. Its mean over the 1 575 rows is the candidate’s validation accuracy.

Then each candidate gets one number (real run)

candidate	validation	train
c1 linear (reference)	79.05 %	80.0 %
c2 tree	75.75 %	83.4 %
c3 forest	78.20 %	100.0 %
c4 MLP	77.40 %	85.0 %

The **leader** is simply the candidate with the best *observed* validation score: here c1. It is not automatically the final answer.

S5 Paired bootstrap — is a difference real or noise?

validation

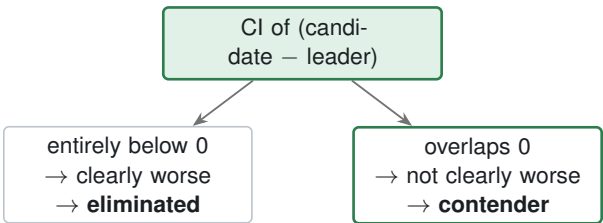


Pairing by row matters: rows that are easy or hard for *every* model cancel out, so the interval reflects how the two candidates actually differ. The uncertainty comes from the finite set of validation rows — not from the seeds. All of this is numpy code; no LLM is involved.

Three comparisons, three different jobs

comparison	question it answers	what it is used for
vs Leader	Is this candidate clearly worse than the best one observed so far?	decides contender status — the only one that does
vs Reference	How does it compare with the original baseline?	context for the Recommender and the UI
vs Parent	Did the latest one-knob change (parent → child) help?	evidence for the next refinement (refined candidates only)

The contender rule (vs Leader)



The leader is always a contender. For MSE (lower is better) “worse” means entirely above 0. An eliminated family is never refined.

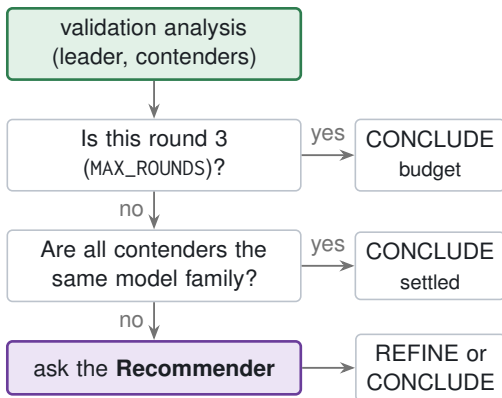
Real run, after round 1 (vs leader c1, percentage points)

candidate	difference	95 % CI → status
c1 linear	—	leader → contender
c2 tree	−3.30	[−5.08, −1.59] → eliminated
c3 forest	−0.85	[−1.90, +0.25] → contender
c4 MLP	−1.65	[−2.88, −0.38] → eliminated

The forest is only 0.85 points behind and that gap is within noise, so it stays in the race. The MLP is only 1.65 points behind, but its interval never reaches 0.

S6 Decide — stop, or ask the Recommender?

Recommender LLM



The two stop rules are code, checked before any LLM call. “Settled” means tuning can no longer change *which family* is best. In the real run the contenders after round 1 were linear and forest — two families — so the Recommender was asked.

The Recommender sees	It never sees
each candidate’s family, parent and change; validation and train score; overfitting gap; seed spread; contender flag; differences + CIs vs reference, leader and parent; the allowed knob ranges of the contenders’ families; rounds left	raw dataset rows; per-row score arrays; anything from the test set; earlier conversations

It receives these as one compact JSON object — about 730 tokens in the real run.

S7 Refine — one contender, one knob, 1–3 values

Recommender LLM

What the Recommender may change

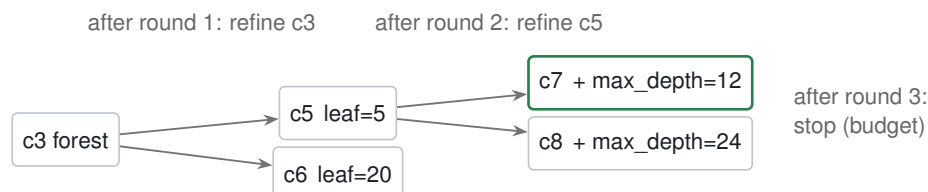
1. pick **one** parent — it must be a contender;
2. pick **one** knob of that parent’s family (e.g. forest: `max_depth`, `min_samples_leaf`, `max_features`);
3. propose **1–3 values**, each inside the knob’s allowed range.

What code checks before anything runs

- all three rules above;
- no new configuration was already tried;
- a violation is sent back to the LLM once; an invalid second answer fails the investigation.

Linear regression has no tunable knobs, so after round 1 the only refinable contender was the forest. A request to refine the MLP (c4) is rejected: “c4 is not a contender”.

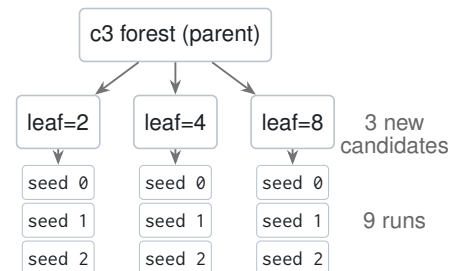
What the real run did (each child’s “vs parent” difference is the effect of one knob)



Round 2: 2 values × 3 seeds = 6 runs. Round 3: 6 runs. Total: 8 candidates, 20 runs. All forest children stayed contenders; none was clearly better than its parent (e.g. c7 vs c5: +0.08 points, CI [−0.38, +0.53]).

Example (illustration with the maximum of 3 values)

parent = c3 forest, knob = `min_samples_leaf`, values = 2, 4, 8



Each child is the parent with *only* `min_samples_leaf` changed. **3 candidates × 3 seeds = 9 runs**, then back to S3: train → validation → bootstrap → new contenders → decide again.

S8 Finalize — the only time the test set is opened

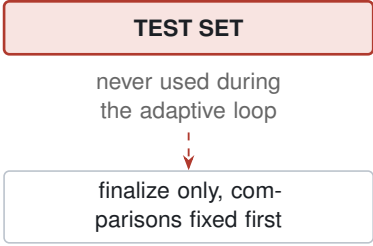
sealed test

- 1. **Winner** W (validation only): the best candidate *other than the reference*. Normally that is the final leader; if the reference itself leads, W is its strongest alternative.
- 2. **Runner-up** R (validation only): the best candidate whose family is neither W 's nor the reference's. If none exists, there is no second comparison.
- 3. **Only now** the sealed test scores of W , the reference and R are read.
- 4. The same paired bootstrap, on the 1 575 test rows: W **vs reference** and W **vs** R .

With k final comparisons, each CI uses (Bonferroni)

$\alpha = \frac{0.05}{k} : \quad k = 2 \Rightarrow \alpha = 0.025, \text{ confidence } 97.5 \%$

$k = 1$ (no runner-up) $\Rightarrow 95 \%$. The validation intervals in S5 are not corrected: they only choose experiments, they are never reported as the answer.



Real run (97.5 % CIs, percentage points)

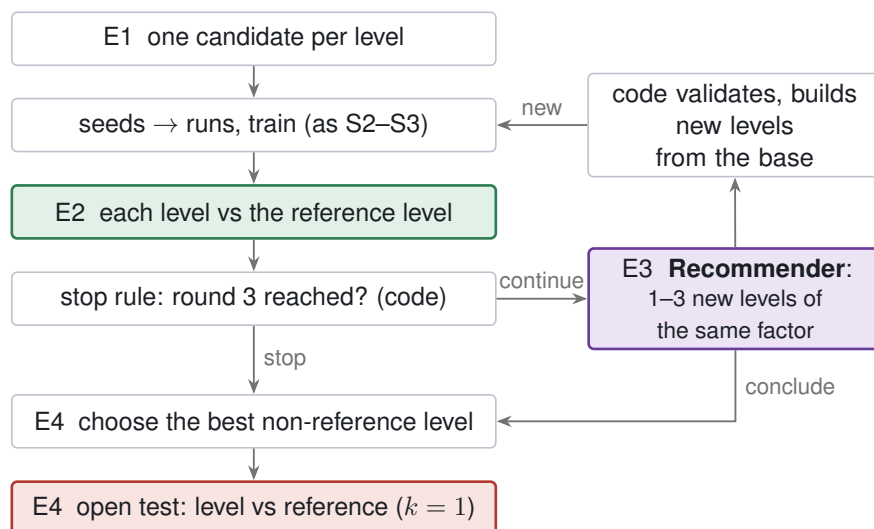
W	c7 forest (79.6 % on test)
reference	c1 linear (80.8 % on test)
R	c4 MLP
W vs reference	$-1.2 [-2.3, -0.1]$ worse
W vs R	$+0.2 [-1.1, +1.6]$ tied

Answer: plain logistic regression beat the tuned forest; the forest and the MLP are statistically tied. The report also lists 4 LLM calls, 20 runs, and how many candidates each family got.

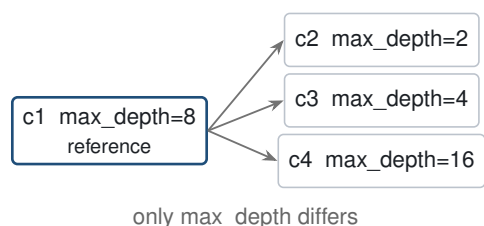
4 Effect Mode — “does factor X matter?”

Running example (real run): “Does tree depth matter for a decision tree on this dataset?” The Planner returns factor = max_depth, levels = 2, 4, 8, 16, reference = 8, base model: decision tree.

Effect Mode does **not** compare model families. Everything is held fixed except one factor, and the question is: *does changing this factor away from the reference level make a measurable difference?* The factor can be model_type, normalize, or any hyperparameter of the base family.



E1 Initial candidates — one per level



c3 compared with c1

same model (decision tree) · same preprocessing · same other hyperparameters
except max_depth = 4 instead of 8

That is what makes the comparison interpretable: any difference is caused by the factor. A decision tree is seed-independent, so each level is **1 run** (4 runs). An MLP factor would give 3 runs per level.

E2 Run and analyse — the same machinery, a simpler comparison

validation

Training, row scores, the sealed test set, seed averaging and the paired bootstrap are exactly S2–S5. The difference: every level is compared with the **reference level** only. There are no leader comparisons, no contenders and no “settled” rule in Effect Mode.

level vs 8	val	difference, 95 % CI
8 (reference)	75.8 %	—
2	73.9 %	−1.8 [−3.8, +0.2]
4	75.6 %	−0.2 [−2.0, +1.5]
16	71.6 %	−4.1 [−6.0, −2.2] worse

Real run, after round 1, percentage points.

The question for each row: *does moving the factor from 8 to this level produce a measurable difference?* Depth 16 clearly hurts (it scores 98 % on its training rows: overfitting); 2 and 4 are within noise of 8.

E3 Refine — more values of the same factor

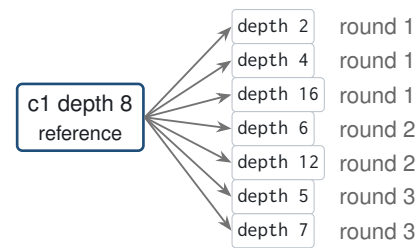
Recommender LLM

The Recommender sees the same kind of compact summary and may only propose **1–3 new values of the same factor**. Code forces the knob to be the factor and the parent to be the reference: every new level is built from the same base configuration, never from another level.

after round 1: “try 6 and 12” → c5, c6

after round 2: “try 5 and 7” → c7, c8

after round 3: stop (budget)



Every level hangs off the reference: one factor, one star.

E4 Finalize — one confirmatory comparison

sealed test

1. From validation: the best non-reference level — here depth 6 (77.0 %).
2. Open the sealed test set once.
3. One comparison: **selected level vs reference level**, so $k = 1$, $\alpha = 0.05$, **95 %** — no Bonferroni division is needed.

Real run — test, 1 575 rows

depth 6: 78.9 % · depth 8: 77.8 %
difference +1.1 points, 95 % CI [−0.3, +2.4]
→ **no reliable difference**
8 runs in 4 s; stopped by the round budget.

5 The two modes side by side

	Selection Mode	Effect Mode
Question	Which model family is best?	Does factor X matter?
Initial design	2–4 model families, each at its defaults	1 factor, 2–5 levels, a base configuration
Reference	the simplest chosen family (code picks it)	the reference level the Planner names
Seeds	3 runs per forest/MLP candidate, 1 per linear/tree candidate	same
Validation comparisons	vs reference, vs leader, vs parent	vs reference only
Contenders	yes: not clearly worse than the leader	none
Stop rules (code)	round 3 reached, or all contenders one family	round 3 reached
Refinement	one contender, one knob of its family, 1–3 values	1–3 new levels of the same factor
Parent of a new candidate	the contender it refines (can be a refined child)	always the reference level
Final comparisons	W vs reference, W vs runner-up from another family	best level vs reference
Test confidence	97.5 % each if 2 comparisons (95 % if 1)	95 %

Hard limits (both modes, all enforced by code): at most 3 rounds · at most 3 new candidates per refinement · MLP epochs ≤ 50 , batch size ≥ 16 · at most 4 LLM calls (Planner, 2 Recommender decisions, 1 explanation) · at most 26 runs (Selection: $8 + 2 \times 3 \times 3$) or 33 runs (Effect: $(5 + 2 \times 3) \times 3$). **Limitation:** every result is conditional on the one fixed training split.